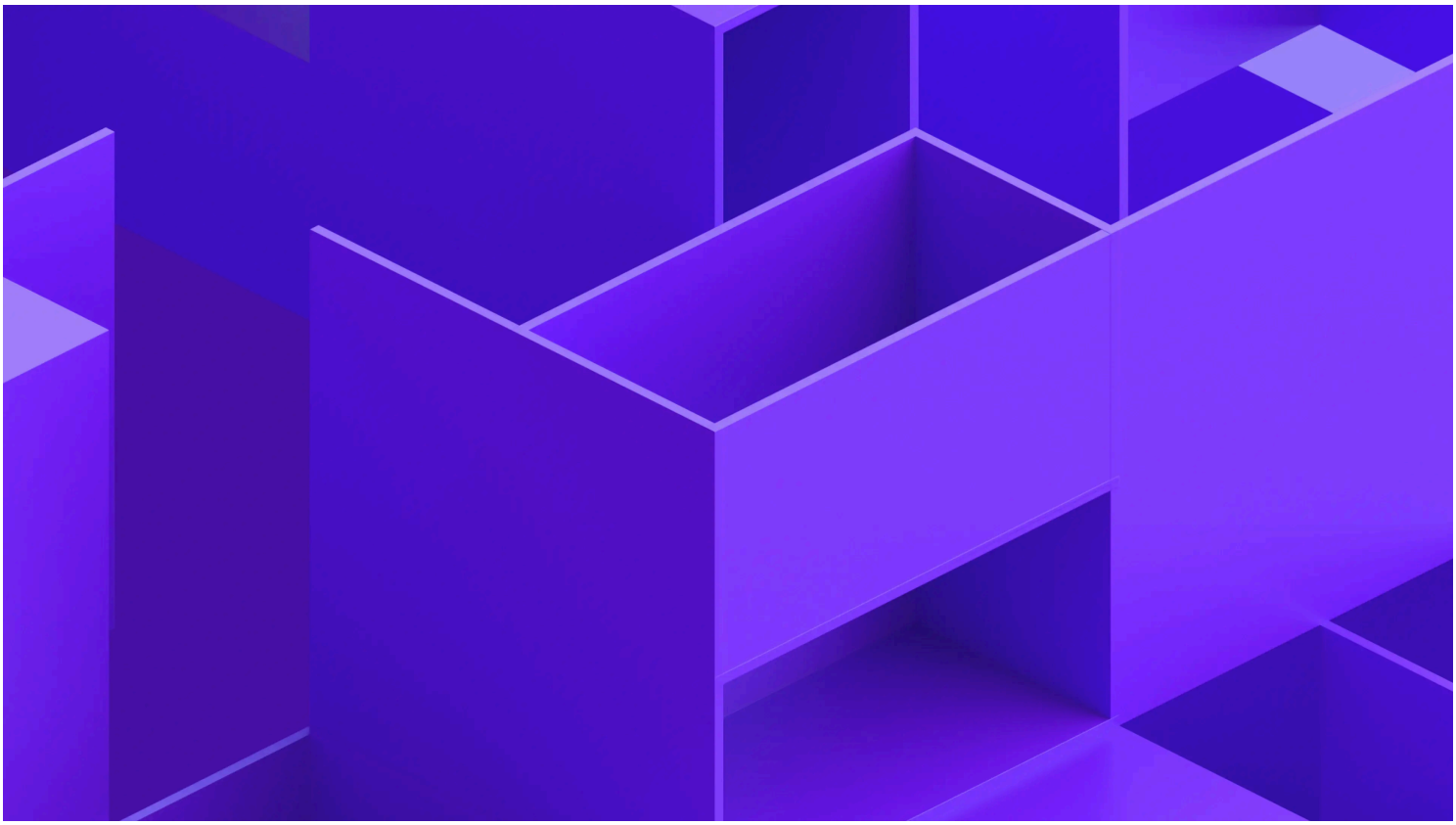
How to Write a Test Strategy Document for Software Testing



N_

13 JUL 2022

SHARE

What is a test strategy document?

A test strategy is a key part of the test process, driven by business requirements. It details the test processes which need to take place to ensure a quality product is developed. It helps to define both test coverage and the scope of testing, ensuring that the team understands the project's scope. It should cover all aspects of the testing process, from manual and automation testing to non-functional requirements (NFRs) such as performance and security testing.

What is the difference between a test strategy and a test plan?

The easiest way to differentiate between them is that a test strategy details the general approach a team should take, whereas a test plan details the specification for carrying out the strategy, by who and when.

Who should create the test strategy?

Typically the test strategy document is created by the project manager or test manager, as they have the most knowledge and exposure to the wider project plan. However as we are seeing more modern and flat team structures, within smaller more agile teams which are made up of mostly senior members, then it can fall upon

the senior test engineer to write the test strategy, gaining approval from project leads to ensure that quality standards for the project are being met.

Can I use a test strategy template?

Each project has different needs and requirements, including different risks. This doesn't mean that you can't start with a base template to accelerate the process, but this should be more about ensuring that different sections are in place, rather than copying the content within those sections. The process of writing a test strategy is more about thinking about risk factors within the project and planning to mitigate those risks, rather than ticking boxes to show that all types of testing have been included.

What should be included in a test strategy document?

The key components of a good test strategy document are as follows:

- **Test objectives and their scope**
- **Key business-led quality requirements**
- **Possible risk factors**
- **Test deliverables**
- **Testing tools**

- Responsibilities
- Tracking and reporting of issues
- **Configuration and change management**
- **Test environment requirements**

It’s important to remember that this isn't a definitive list, if you find something else that is important to your project, then include it.

Keep the above requirements in mind when filling out the following sections, which again are a base recommendation and should guide, rather than dictate, how you create your test strategy.

1. Version History and Sign Offs

Any valuable project document should have a version history table and document sign-off table at the top of the document to ensure the quality of the document over time.

Version History

This table details each major change that is made to the test strategy document. Usually, there’s a draft and then a released version which is signed off. Then any further changes to this document will increment the release number and will also need to be signed off again.

Version	Purpose / Change	Author	Date
0.1	Draft	Joe Bloggs	11/07/2022
1.0	Release	Joe Bloggs	15/07/2022

Document Sign Off

Once the test strategy is complete and released it needs to be signed off by key project stakeholders as it becomes a contract of quality. Any further changes after that will also need to be signed off to ensure that the high-quality standards are adhered to.

Name	Date	Version
Mr Smith	16/07/2022	1.0

2. Objectives

List the objectives of the test strategy, detailing what you want to achieve as a result of following it.

Example:

- Determine which test types are required
- Maximize test coverage and minimize duplication of effort
- Support continuous deployment
- To achieve expected standards of non-functional requirements
- To meet industry standards

3. Benefits

List the benefits which you want to achieve.

Example:

- Earlier identification of testing requirements
- Promote communication between team members
- Mitigation of risk
- Better understanding of test tasks

- Recognise areas of responsibility

Nearform

4. Scope

List the items which fall into the scope of this document. You can also list ones that explicitly don't too.

Example:

In Scope

- Unit Tests
- Integration / E2E / UI Tests
- Acceptance Tests
- API Tests
- Security Tests
- Performance Tests
- Browser Tests
- Accessibility Tests
- Automation of Tests
- Identify and agree on area of responsibility for developers and testers
- Requirements source: NFRs

Out of Scope

- Time scales
- Effort
- Cost

5. Types of Testing

This is a more detailed section and has sub-headings for each type of testing which is to be carried out. Also, feel free to throw in a test pyramid if you like.

I've found that it's best to break down each type of testing and detail the following:

- Challenges / Benefits
- Strategy
- Specific Requirements

Here's an example of one of the sections you could include:

Manual and Exploratory Tests

Challenges / Benefits

- Early identification of bugs
- Being able to test areas which are not part of the happy path
- Use the system in unexpected ways to ensure it still functions as expected
- Expensive to perform

Strategy

Strategy	Stage	By Who
Manual testing	Ongoing	Primarily by test engineers, but also by developers as needed (whole team testing effort)
Exploratory testing	Ongoing (Timeboxed)	All members of the team should take time to think outside of the box and do some time-boxed testing

Specific Requirements

- Manual tests logged against stories / tasks
- Bugs logged in tracking software
- Regression test of fixed bugs to be performed
- (Link to specific NFRs)

And here's a list of sections that you may want to include:

- Manual and Exploratory Tests
- UI Automation Tests
- API Tests
- Integration Tests
- User Acceptance Tests
- Performance Tests
- Security Tests
- Browser Compatibility Tests
- Accessibility Tests

6. Test Environments

List the different test environments which will be needed.

Example:

Environment	Requirement
Local	Local dev env needed to fetch latest build, execute tests and write test automation scripts

Test	Latest code snapshot of dev branch taken daily
Demo	Handpicked deployment to showcase
Regression	Obfuscated customer data and client configurations
UAT	Client-facing environment, used by client for UAT

7. Test Data

List the data requirements that will be needed to satisfy the different stages of testing.

Example:

Timing	**Data Requirements**
Milestone 1	Mock data (1,000 records)
Milestone 2	Obfuscated data (max 5,000 records)
Milestone 3	Large scale obfuscated data (1M records)

8. Testing Tools

List the different testing tools that you will require to carry out testing. Also note if tools are yet to be decided on so that investigation spikes can happen as early as possible.

Example:

Tool	Purpose
Cypress	UI automation
Autocannon	Load testing
Puppeteer	Frontend performance testing
Postman	API tests
Jest	Frontend unit tests
Node Tap	Backend unit tests
TBD	Security testing
TBD	Accessibility testing

9. Test Results

List any test reports which will be output and detail any retention policies. Also, list any reporting tools / plugins that you will use. Such as the Zephyr plugin for Jira which is a popular way of documenting manual tests or Report Portal which is an open-source test reporting solution.

10. Defect Tracking

Identify how bugs will be logged and tracked, such as with GitLab. Also, detail how bugs will be triaged if necessary.

11. Risks

Try to identify and list any risks which may occur during this project, detailing any steps that can mitigate these risks. You can also detail any contingency plans in case the risks become reality.

Conclusion

Please remember that this is a guide, not a definitive list of what goes into a test strategy document. Each project is different and every team will have different needs. Also remember that you can update the test strategy document as and when needed, with approval of course, as we all know that project requirements can change.

Insight, imagination and expertly engineered solutions to accelerate and sustain progress.

Contact

N_

Nearform is an independent team of data & AI experts, engineers, and designers who build intelligent digital solutions and capability at pace.

© 2026 Nearform